State Estimation in Driverless Vehicles

Thejas Padubidri Rao
Driverless Taskphase
(of Affiliation)
Formula Manipal
(of Affiliation)
Manipal, Karnataka
xxxxxxxx@gmail.com

***Abstract*—An autonomous or driverless vehicle (DV) never has direct access to its true kinematic state or the exact states of surrounding obstacles. Wheel slip, unmodeled road gradients, and actuator latencies corrupt motion predictions, while thermal noise, multipath reflections, and quantization degrade exteroceptive and proprioceptive sensor measurements. Recursive Bayesian filtering provides a rigorous probabilistic framework for fusing imperfect motion models with noisy sensor observations over time. This report presents a comprehensive derivation and pedagogical analysis of the general Bayes filter, establishes the three foundational assumptions under which it reduces to the closed-form linear Kalman filter, examines the five governing equations of the Kalman filter alongside the asymptotic behavior of the Kalman gain, and details how Kalman filtering integrates into a full-stack driverless vehicle pipeline.**

Keywords—Bayes Filter, Kalman filter

I. Introduction

State estimation is the study of reproducing the hidden internal state of a robot or vehicle (such as its location, velocity, or orientation) from noisy sensor measurements. Every measured or computed parameter is an estimate characterized by accuracy (how close the measurement mean is to the true value, where systematic offsets represent bias) and precision (the dispersion or variability of repeated measurements, quantified by random measurement noise variance σ^2). In an unbiased system, random measurement errors fluctuate above and below the true state, while dynamic motion models experience unmodeled fluctuations represented by process noise.

In discrete time, filtering and state estimation are formulated using Linear Dynamical Systems (LDS). For a system with state vector $x_t \in \mathbb{R}^n$ evolving over discrete timesteps $t \in \{1, 2, \dots\}$ under control input u_t and sensor output $y_t \in \mathbb{R}^p$:

$$x_{t+1} = A_t x_t + B_t u_t + c_t \qquad (1)$$

$$y_t = C_t x_t + D_t u_t \qquad (2)$$

where $A_t \in \mathbb{R}^{n \times n}$ is the dynamics matrix, B_t is the control input matrix, c_t is an offset vector, $C_t \in \mathbb{R}^{p \times n}$ is the output (sensor) matrix, and D_t is the direct feedthrough matrix. An LDS is a special case of a Markov system in which the next state is a linear function of the current state. In a probabilistic setting, we are given:

1. A measurement distribution $p(y_t, \mid x_t u_t)$ (or $p(y_t \mid x_t)$ when measurements depend only on the current state), and
2. A state transition distribution $p(x_{t+1}, \mid x_t u_t)$ (or $p(x_t \mid x_{t-1})$) representing the robot or vehicle dynamics.

The overarching objective of state estimation is to compute the posterior distribution over the hidden state given all prior measurements $y_{1:t} = (y_1, y_2, \dots, y_t)$ and inputs $u_{1:t}$:

$$p(x_t, \mid y_{1:t} u_{1:t}) \qquad (3)$$

II. The Recursive Bayes Filter

A. Probability Foundations: Chain Rule, Marginalization, and Bayes' Rule:

Three foundational rules of probability are required to derive recursive estimators:

1) The Chain Rule: For events $y_1, y_2, \dots, y_t$ with $p(y_i) > 0$:

$$p(y_1 \cap y_2 \cap \cdots \cap y_t) = p(y_1)p(y_2 \mid y_1) \cdots p(y_t \mid y_1 \cap \cdots \cap y_{t-1})$$

2) Marginalization (Variable Elimination): A joint distribution over (X_1, X_2) can be integrated over x_2 to eliminate X_2:

$$p(x_1) = \int_{x_2} p\,(x_1, x_2)\, dx_2 \qquad (5)$$

3) Bayes' Rule in State Estimation: For state x and measurement y with $p(x) > 0$ and $p(y) > 0$:

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{p(y)} = \frac{p(y \mid x)p(x)}{\int_x p\,(y \mid x)p(x)\, dx} \qquad (6)$$

where $p(x)$ is the prior, $p(y \mid x)$ is the measurement likelihood, $p(y) = \int_x p\,(y \mid x)p(x)\, dx$ is the normalization term, and $p(x \mid y)$ is the posterior (what we cannot see, conditioned on what we can see).

B. Static Bayesian Estimation vs. Dynamic Bayesian Filtering

Recursive inference problems fall into two distinct classes depending on whether the underlying state is static or time-varying:

1) Bayesian Estimation (Static State X): The underlying state X is static while a sequence of noisy measurements $(Y_1, Y_2, \dots, Y_t)$ is revealed over time. This corresponds to a Naive Bayes graphical model where

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

measurements Y_i are conditionally independent given X. Applying Bayes' rule and factoring the likelihood yields the recursive loop Measure → Estimate → Measure → Estimate:

$$p(x \mid y_{1:t}) = \frac{p(y_t \mid x)p(x \mid y_{1:t-1})}{\int_x p(y_t \mid x)p(x \mid y_{1:t-1})\,dx} \quad (7)$$

Where the posterior $p(x \mid y_{1:t-1})$ from timestep $t-1$ directly serves as the prior at timestep t.

2) Bayesian Filtering (Dynamic State X_t): The underlying state evolves over time ($X_1 \rightarrow X_2 \rightarrow \cdots \rightarrow X_t$). This system is modeled graphically as a Hidden Markov Model (HMM) because the true states X_t are hidden and must be inferred from noisy observations Y_t.

The HMM structure provides two structural independence properties that make recursive filtering possible without storing an infinite history:

1. The Markov Property (Markovianity): Given the previous state x_{t-1}, the current state x_t is conditionally independent of all earlier states and measurements:

$$p(x_t, \mid x_{1:t-1}y_{1:t-1}) = p(x_t \mid x_{t-1}) \quad (8)$$

2. Conditional Independence of Measurements: Given the current state x_t, the current measurement y_t is conditionally independent of all past states and measurements:

$$p(y_t, \mid x_{1:t}y_{1:t-1}) = p(y_t \mid x_t) \quad (9)$$

At each timestep t, the Bayes filter takes the prior belief from $t-1$ and executes a two-step Predict–Update cycle (written below with control inputs omitted for compactness, as in standard HMM notation):

a) Step 1- Predict Step (Time / Motion Update): Before incorporating the new sensor reading y_t, the filter predicts the state distribution at timestep t given only prior measurements $y_{1:t-1}$. Applying marginalization over x_{t-1}, the chain rule of probability, and the Markov property yields:

$$p(x_t \mid y_{1:t-1}) = \int_{x_{t-1}} p(x_t \mid x_{t-1})\, p(x_{t-1} \mid y_{1:t-1})\, dx_{t-1} \quad (4)$$

Physically, this step shifts and spreads the belief distribution forward in time according to the vehicle's motion model. Because process noise enters during motion, state uncertainty increases during prediction.

b) Step 2 - Update Step (Measurement Update): When sensor observation y_t arrives, the filter sharpens the predicted belief using Bayes' rule and the conditional independence of measurements:

$$p(x_t \mid y_{1:t}) = \frac{p(y_t \mid x_t)\, p(x_t \mid y_{1:t-1})}{\int_{x_t} p(y_t \mid x_t)\, p(x_t \mid y_{1:t-1})\, dx_t} = \eta\, p(y_t \mid x_t)\, p(x_t \mid y_{1:t-1}) \quad (5)$$

Physically, this step multiplies the predicted belief by the sensor measurement likelihood and normalises the result so the updated posterior integrates to 1.

III. Meaning of Every Term in the Bayes Filter

Term	Mathematical Expression	Physical Meaning and Role in the Filter
Prior Belief	$p(x_{t-1} \mid y_{1:t-1})$	The posterior probability distribution over the vehicle's state from timestep $t-1$, summarizing all information prior to moving at step t.
Motion Model	$p(x_t \mid x_{t-1})$	The state transition probability distribution. Describes vehicle kinematics/dynamics and process noise over $[t, -1t]$.
Predicted Belief	$p(x_t \mid y_{1:t-1})$	The *a priori* belief over state x_t computed in the Predict Step after propagating the prior belief through the motion model, before observing y_t.
Measurement Likelihood	$p(y_t \mid x_t)$	The probabilistic sensor model. Quantifies how likely it is to observe sensor reading y_t if the vehicle were truly at state x_t.
Normaliser	$\eta=[\int x_t p(y_t \mid x_t) p(x_t \mid y_{1:t-1}) dx_t]^{-1}$	Reciprocal of the marginal measurement probability $p(y_t \mid y_{1:t-1})$. Scales the product of likelihood and predicted belief so the posterior integrates to 1.
Posterior Belief	$p(x_t \mid y_{1:t})$	The updated (*a posteriori*) belief over x_t after fusing the motion prediction with sensor measurement y_t. Becomes the prior belief for step $t+1$.

IV. Reduction of the Bayes Filter to the Kalman Filter

For arbitrary continuous probability distributions, the integrals in Equations (4) and (5) cannot be solved in closed form. However, under three specific assumptions, the Bayes filter reduces to an exact, closed-form analytical solution: the Linear Kalman Filter (LKF). The Three Core Assumptions

Using standard discrete-time Kalman filter notation at steps n:-

A. Linear System Dynamics and Linear Measurement Models: Both the state extrapolation equation and the state-to-measurement relation are linear transformations:

$$x_{n+1} = Fx_n + Gu_n + w_n \qquad (6)$$

$$z_n = Hx_n + v_n \qquad (7)$$

where F is the state transition matrix, G is the control matrix, u_n is the control input vector, z_n is the measurement vector, and H is the observation matrix.

B. Unbiased (Zero-Mean) Gaussian Process and Measurement Noise: The random process noise vector $\mathrm{w_n}$ and measurement noise vector $\mathrm{v_n}$ are unbiased, white, zero-mean multivariate Gaussian random variables:

$$\mathrm{w_n} \sim \mathcal{N}(0, \mathrm{Q}),\ \mathrm{v_n} \sim \mathcal{N}(0, \mathrm{R_n}) \qquad (8)$$

where $\mathrm{Q} = \mathrm{E}(\mathrm{w_n w_n^T})$ is the process noise covariance matrix and $\mathrm{R_n} = \mathrm{E}(\mathrm{v_n v_n^T})$ is the measurement noise covariance matrix.

C. Gaussian Initial Belief: The initial belief over the state at step $\mathrm{n} = 0$ is a unimodal multivariate Gaussian distribution characterized by mean vector $\hat{\mathrm{x}}_{0,0}$ and covariance matrix $\mathrm{P}_{0,0}$.

A multivariate Gaussian distribution is completely determined by its first two moments: its mean vector $\hat{\mathrm{x}}$ and its covariance matrix P. The three assumptions above guarantee that the belief remains strictly Gaussian at every step of the recursion:

1) Closure Under Linear Extrapolation (Predict Step): Passing a Gaussian random vector $\mathrm{x_n} \sim \mathcal{N}(\hat{\mathrm{x}}_{\mathrm{n,n}}, \mathrm{P_{n,n}})$ through a linear transformation F and adding an independent Gaussian noise vector $\mathrm{w_n} \sim \mathcal{N}(0, \mathrm{Q})$ produces another exact Gaussian distribution. By the linearity of expectation and covariance ($\mathrm{COV(Fx)} = \mathrm{FPF^T}$), the marginalization integral in Eq. (4) reduces to simple matrix multiplication: $\hat{\mathrm{x}}_{\mathrm{n+1,n}} = \mathrm{F}\hat{\mathrm{x}}_{\mathrm{n,n}} + \mathrm{Gu_n}$ and $\mathrm{P_{n+1,n}} = \mathrm{FP_{n,n}F^T} + \mathrm{Q}$.

2) Closure Under Multiplication of Gaussians (Update Step): In Eq. (5), multiplying the Gaussian predicted belief PDF by the Gaussian measurement likelihood PDF adds two quadratic forms in the exponent, producing a new Gaussian PDF. The updated mean is an optimal weighted average of the prediction and measurement, and the normaliser integral η never has to be evaluated numerically

D. What Goes Wrong When the Assumptions Do Not Hold

1. Violation of Linearity (Non-Linear Dynamics or Sensors): In real vehicles, motion models involve trigonometric heading terms ($\cos\psi$, $\sin\psi$) and sensors such as radar or LiDAR measure polar range and bearing (r, ϕ) rather than Cartesian coordinates (x, y). Passing a Gaussian distribution through a non-linear function warps its shape so the output is no longer Gaussian.

a) Approximating the system via a first-order Taylor series / Jacobian matrix around the current mean (Extended Kalman Filter, EKF) can introduce severe linearization errors if the function is highly curved, causing covariance underestimation and filter divergence.

b) Strongly non-linear systems require statistical linearization using deterministic sigma points via the Unscented Transform (Unscented Kalman Filter, UKF).

2. Violation of Unbiased Gaussian Noise: The Kalman filter assumes zero-mean Gaussian noise where positive and negative errors average out and large outliers are exponentially unlikely. If a sensor has a systematic offset (bias), the state estimate converges to an inaccurate, biased value. If process noise covariance Q is set too small compared to true vehicle dynamics, the filter suffers from estimation lag error.
3. Violation of Unimodal Gaussian Belief: A Gaussian PDF has only a single peak (mode). If a vehicle faces discrete multi-modal ambiguity (such as two distinct candidate locations or ambiguous landmark associations), a single Gaussian places its mean between the modes—often in an invalid state—rather than tracking separate hypotheses.

V. The Kalman Filter Equations and Kalman Gain Analysis

At each discrete cycle n , the Linear Kalman Filter executes five matrix equations split across a Prediction (Extrapolation) phase and a Measurement Update (Correction) phase. Here, subscript (n, n) denotes the updated estimate at step n , $(n, +1n)$ denotes the prediction for step $n + 1$ made at step n, and $(n, n - 1)$ denotes the prior prediction for step n carried over from step $n - 1$.

VI. Step-by-Step Walkthrough of the Five Equations

A. Phase 1: Prediction Step (Extrapolation)

$$\hat{\mathrm{x}}_{\mathrm{n+1,n}} = \mathrm{F}\hat{\mathrm{x}}_{\mathrm{n,n}} + \mathrm{Gu_n} \qquad (10)$$

$$\mathrm{P_{n+1,n}} = \mathrm{FP_{n,n}F^T} + \mathrm{Q} \qquad (11)$$

1) Equation (10) — State Extrapolation Equation: Projects the current state estimate $\hat{\mathrm{x}}_{\mathrm{n,n}}$ forward to the next timestep using the state transition matrix F and deterministic control input $\mathrm{Gu_n}$. Because process noise has zero mean, it does not appear in the state mean prediction.

2) Equation (11) — Covariance Extrapolation Equation: Projects the current estimate uncertainty $\mathrm{P_{n,n}}$ forward through the system dynamics via $\mathrm{FP_{n,n}F^T}$ and adds the process noise covariance matrix Q, reflecting the growth of uncertainty during motion without new measurements.

B. Phase 2: Measurement Update Step (Correction)

$$\mathrm{K_n} = \mathrm{P_{n,n-1}H^T}\left(\mathrm{HP_{n,n-1}H^T} + \mathrm{R_n}\right)^{-1} \qquad (12)$$

$$\hat{\mathrm{x}}_{\mathrm{n,n}} = \hat{\mathrm{x}}_{\mathrm{n,n-1}} + \mathrm{K_n}\left(\mathrm{z_n} - \mathrm{H}\hat{\mathrm{x}}_{\mathrm{n,n-1}}\right) \qquad (13)$$

$$P_{n,n} = (I - K_n H)P_{n,n-1}(I - K_n H)^T + K_n R_n K_n^T \quad (14)$$

1) Equation (12) — Kalman Gain Equation: Computes the optimal weighting matrix K_n that minimizes the posterior estimate variance. It forms the ratio between the prior state covariance projected into measurement space ($P_{n,n-1}H^T$) and the total expected measurement uncertainty ($HP_{n,n-1}H^T + R_n$).

2) Equation (13) — State Update Equation: Corrects the prior prediction $\hat{x}_{n,n-1}$ by adding a correction proportional to the innovation (measurement residual) $(z_n - H\hat{x}_{n,n-1})$, which measures the discrepancy between the actual sensor reading z_n and the expected sensor reading $H\hat{x}_{n,n-1}$. Rewriting Eq. (13) as $\hat{x}_{n,n} = (I - K_n H)\hat{x}_{n,n-1} + K_n z_n$ shows that the updated state is a weighted average of the prediction and measurement.

3) Equation (14) — Covariance Update Equation: Reduces the prior uncertainty $P_{n,n-1}$ by the factor $(I - K_n H)$ to reflect the information gained from measurement z_n. While the simplified form $(I - K_n H)P_{n,n-1}$ is compact, the full symmetric Joseph form on the left of Eq. (14) is used in practical software to prevent floating-point subtraction roundoff errors from destroying matrix symmetry.

VII. Behavior of the Kalman Gain: Very Noisy vs. Very Accurate Sensor

In one dimension ($H = 1$), minimizing the posterior variance $p_{n,n} = w_1^2 r_n + (1 - w_1)^2 p_{n,n-1}$ with respect to weight w_1 ($dp_{n,n}/dw_1 = 0$) yields the scalar Kalman gain:

$$K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n},\ 0 \le K_n \le 1 \quad (15)$$

where $p_{n,n-1}$ is the prior prediction variance and r_n is the sensor measurement noise variance:

1. When the Sensor is Very Noisy ($r_n \gg p_{n,n-1}$ or $R_n \to \infty$):

$$\lim_{R_n \to \infty} K_n = 0 \left(\text{in 1D: } K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n} \to 0 \right) \quad (16)$$

Substituting $K_n = 0$ into Equations (13) and (14) gives:

$$\hat{x}_{n,n} = \hat{x}_{n,n-1},\ P_{n,n} = P_{n,n-1} \quad (17)$$

Physical Meaning: When measurement uncertainty is much larger than prediction uncertainty, the Kalman gain approaches zero. The filter completely ignores the noisy sensor measurement z_n, trusts its internal motion model prediction $\hat{x}_{n,n-1}$ completely, and leaves the estimate uncertainty unchanged. 2. When the Sensor is Very Accurate ($r_n \ll p_{n,n-1}$ or $R_n \to 0$):

$$\lim_{R_n \to 0} K_n = H^{-1} \left(\text{in 1D with } H = 1\text{: } K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n} \to 1 \right)$$

Substituting $K_n = H^{-1}$ (or $K_n = 1$) into Equations (13) and (14) gives:

$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + H^{-1}(z_n - H\hat{x}_{n,n-1}) = H^{-1}z_n \text{ (or } z_n),\ P_{n,n} = 0$$

A. *Physical Meaning:* When the sensor is virtually noiseless (or when prior uncertainty $p_{n,n-1}$ is huge during initialization), the Kalman gain approaches unity (H^{-1}). The filter discards the motion prediction, snaps the posterior state estimate directly to the sensor reading, and collapses the posterior uncertainty toward zero.

VIII. Relevance to Our Car: Where the Kalman Filter Fits in the DV Pipeline

In our Driverless Vehicle (DV) software architecture (Sensors → Perception → State Estimation & Mapping → Path Planning → Motion Control), Kalman filters operate at two critical junctions:

A. *Ego-Vehicle State Estimation:* Sitting between the onboard sensors and the control modules, the filter fuses high-frequency inertial measurements (IMU) with slower absolute position fixes (such as GPS or SLAM). During the prediction step, IMU acceleration data estimates immediate vehicle movement. When a position fix arrives, the update step corrects the accumulated inertial drift. This provides the steering and speed controllers with smooth, low-latency position and velocity estimates without dead-reckoning drift.

B. *Perception Back-End and Object Tracking:* Located between raw perception detections (from LiDAR or cameras) and the mapping module, a second Kalman filter tracks detected objects in planar (X, Y) space. Although the sensors only measure raw object positions, the filter uses a kinematic model to suppress detection jitter and infer hidden states—specifically velocity and acceleration—across consecutive frames. This supplies the Path Planning module with stable, full-state trajectories of surrounding obstacles.

Acknowledgments

I would like to express my gratitude to FORMULA MANIPAL, for the assignment that prompted this deep dive into state estimation and Kalman filters.

References

[1] J. W. Lambert, "The Bayes Filter," GitHub Pages Technical Notes. Available: https://johnwlambert.github.io/bayes-filter/.

[2] A. Becker, Kalman Filter from the Ground Up. Available: https://kalmanfilter.net/.